

Given an array $arr[]$ containing both positive and negative integers, check whether there exists a non-empty subarray whose sum is equal to zero.

input $arr = [4, 2, -3, 1, 6]$ $\rightarrow$ True

input $arr = [-3, 2, 3, 1, 6]$ $\rightarrow$ False

$[2, -3, 1]$

Brut Force

Generate every subarray using two nested loops and calculate the sum of each subarray. Check if subarray sum is 0 then return true. Otherwise, if no such subarray found then return false.

3	-2	1	1	4
0	1	2	3	4

Sum = ~~3~~ ~~1~~ ~~2~~ ~~3~~ ~~7~~
= ~~3~~ ~~1~~ 0 True

Better Approach

[Expected Approach] Using Hashing - $O(n)$ Time and $O(n)$ Space

The idea is to iterate through the array and for every element $arr[i]$, calculate the sum of elements from 0 to i (this can simply be done as $sum += arr[i]$). If the current sum has been seen before, then there must be a zero-sum subarray. Hashing is used to store the sum values so that sum can be stored quickly and find out whether the current sum is seen before or not.

mathematical proof.

Let S_i be the sum of all elements from the beginning of the array upto the index i .

$$S_i = arr[0] + arr[1] + \dots + arr[i]$$

Let, S_j be the sum from 0 to j .

$$S_j = arr[0] + arr[1] + \dots + arr[i] + arr[i+1] + \dots + arr[j]$$

$$\rightarrow S_j = S_i + (arr[i+1] + \dots + arr[j])$$

Now if $S_i = S_j$ then

$$arr[i+1] + \dots + arr[j] = 0$$

3	4	-2	-2	1
---	---	----	----	---

index 0 : 3 $\rightarrow$ 3
 index 1 : 3+4 $\rightarrow$ 7
 index 2 : 3+4+(-2) $\rightarrow$ 5
 index 3 : 3+4+(-2)+(-2) $\rightarrow$ 3
 index 4 : 3+4+(-2)+(-2)+1 $\rightarrow$ 4

Same

Corner Case : The whole prefix can also be 0, so along with repetition of prefix sum, we also need to check if prefix sum becomes 0.

class Solution:

def subArrayExists(self, arr):

sumSet=set()

sum=0

for i in arr:

sum+=i

if sum==0 or sum in sumSet:

return True

else:

sumSet.add(sum)

return False

python
code

```

#include<bits/stdc++.h>
using namespace std;

// O(N^2) Time Complexity
/* bool subarrayWithSumZero(vector<int>& arr){
    int n=arr.size();
    for(int i=0;i<n;i++){
        int sum=arr[i];
        if(sum==0){
            return true;
        }
        for(int j=i+1;j<n;j++){
            sum+=arr[j];
            if(sum==0){
                return true;
            }
        }
    }
    return false;
} */

bool subarrayWithSumZero(vector<int>& arr){
    unordered_set<int> sumSet;
    int sum=0;
    for(int i=0;i<arr.size();i++){
        sum+=arr[i];
        if(sum==0 || sumSet.find(sum)!=sumSet.end()){
            return true;
        }else{
            sumSet.insert(sum);
        }
    }
    return false;
}

int main(){
    // vector<int> arr={3,-2,1,1,4};
    vector<int> arr={3,2,1,1,4};
    if(subarrayWithSumZero(arr)){
        cout<<"True"<<endl;
    }else{
        cout<<"False"<<endl;
    }
    return 0;
}

```

*C++
Code*